



MODULE 03/09

Node.js & Express

Construire un backend API REST solide

■ Durée : 1 semaine

■ Projet : Social Seller

■ Niveau : Débutant →
Opérationnel

Objectifs de la semaine

- ✓ Comprendre comment Node.js exécute du JavaScript côté serveur
 - ✓ Créer un serveur HTTP avec Express et définir des routes
 - ✓ Implémenter des middlewares pour l'authentification et la validation
 - ✓ Structurer une API REST : routes, contrôleurs, services
 - ✓ Déployer une API Express sur Railway comme Social Seller
-

Planning de la semaine

JOUR 1	Node.js et Event Loop	2-3h
• Node.js : JavaScript sans navigateur, event loop non-bloquant • npm / node_modules : gérer les dépendances • Modules CommonJS vs ESM modules (import/export) • fs, path, crypto — les modules natifs essentiels		
JOUR 2	Express — routes et middleware	2-3h
• Créer un serveur Express basique en 5 lignes • Routes GET/POST/PATCH/DELETE avec paramètres et query strings • Middleware : <code>express.json()</code>, <code>cors()</code>, <code>helmet()</code> • Chaîne de middleware : <code>next()</code> et gestion des erreurs		
JOUR 3	Validation et erreurs	2-3h
• Valider les inputs avec Zod (comme dans Social Seller) • Classes d'erreurs personnalisées (<code>AppError</code>, <code>NotFoundError</code>...) • Middleware d'erreur global : (<code>err</code>, <code>req</code>, <code>res</code>, <code>next</code>) • Codes HTTP : 200, 201, 400, 401, 403, 404, 409, 429, 500		
JOUR 4	Auth JWT et structure projet	2-3h
• JWT : <code>header.payload.signature</code> — comment ça fonctionne • Middleware <code>requireAuth</code> : vérifier le token à chaque requête • Structure projet : <code>routes/</code>, <code>middleware/</code>, <code>services/</code>, <code>lib/</code> • Variables d'environnement avec <code>dotenv</code>		
JOUR 5	Mini-projet de la semaine	3-4h
• Créer une API REST 'Todo List' complète avec Express + TypeScript • Routes : GET <code>/todos</code>, POST <code>/todos</code>, PATCH <code>/todos/:id</code>, DELETE <code>/todos/:id</code>		

- Validation Zod sur chaque route POST/PATCH
- Middleware auth JWT basique (token statique pour l'exercice)
- Gestion d'erreurs globale avec classes d'erreur custom

Concepts clés détaillés

Express en 5 lignes

```
import express from 'express';

const app = express();
app.use(express.json()); // Parser le body JSON

app.get('/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString() });
});

app.listen(3000, '0.0.0.0', () => {
  console.log('Serveur démarré sur le port 3000');
});

// Note Railway : ne jamais forcer PORT=3000
// Utiliser : const PORT = process.env.PORT ?? 3000;
```

Routes et paramètres

```
const router = express.Router();

// Paramètre de route : :id
router.get('/:id', async (req, res, next) => {
  const { id } = req.params; // /conversations/abc-123 → id = 'abc-123'
  // ...
});

// Query string : ?limit=20&offset=0
router.get('/', async (req, res, next) => {
  const limit = Number(req.query.limit) || 20;
  const offset = Number(req.query.offset) || 0;
  // ...
});

// Body JSON : POST avec payload
```

```
router.post('/', async (req, res, next) => {  
  const { content, direction } = req.body;  
  // ...  
});
```

Middleware — la brique modulaire

```
// Un middleware = (req, res, next) => void  
// Il peut : modifier req/res, répondre, ou passer à next()  
  
// Exemple : logger  
function logger(req, res, next) {  
  console.log(`${req.method} ${req.path}`);  
  next(); // Passer au middleware suivant  
}  
  
// Exemple : auth JWT (simplifié)  
function requireAuth(req, res, next) {  
  const token = req.headers.authorization?.slice('Bearer '.length);  
  if (!token) {  
    res.status(401).json({ error: 'Token manquant' });  
    return; // Stopper la chaîne  
  }  
  // Vérifier le token...  
  req.user = { id: '...', tenantId: '...' };  
  next();  
}  
  
// Middleware d'erreur global (4 paramètres obligatoires)  
function errorHandler(err, req, res, next) {  
  const status = err.statusCode || 500;  
  res.status(status).json({ error: err.publicMessage || 'Erreur interne' });  
}  
app.use(errorHandler); // TOUJOURS en dernier
```

Validation avec Zod

```
import { z } from 'zod';  
  
const createMessageSchema = z.object({  
  content: z.string().trim().min(1).max(4096),
```

```
direction: z.enum(['inbound', 'outbound']),
});

router.post('/', async (req, res, next) => {
  const parsed = createMessageSchema.safeParse(req.body);
  if (!parsed.success) {
    next(new ValidationError(parsed.error.issues[0]?.message));
    return;
  }
  // parsed.data est typé et validé
  const { content, direction } = parsed.data;
  // ...
});
```

■ *Structure de projet Social Seller : backend/src/routes/ (définition des routes), backend/src/middleware/ (auth, rate limiting), backend/src/services/ (logique métier réutilisable), backend/src/lib/ (utilitaires : logger, erreurs, crypto).*

Exercices pratiques

Exercice 1 — Serveur basique (20 min)

```
# Créer un projet Node + TypeScript + Express
mkdir api-todo && cd api-todo
npm init -y
npm install express zod
npm install -D typescript @types/express tsx
npx tsc --init
```

Exercice 2 — CRUD complet (1h)

- Implémenter GET /todos (retourner un tableau en mémoire)
- POST /todos avec validation Zod (title: string requis)
- PATCH /todos/:id pour marquer comme fait
- DELETE /todos/:id avec soft delete (deletedAt)

Exercice 3 — Middleware auth (40 min)

- Ajouter un middleware qui vérifie un header X-API-Key
 - Retourner 401 si absent, 403 si invalide
 - Protéger toutes les routes sauf GET /health
-

Ressources pour aller plus loin

- Documentation Express : expressjs.com/fr
- Zod : zod.dev
- Railway (déploiement) : docs.railway.app
- Fastify (alternative rapide à Express) : fastify.dev
- Node.js best practices : github.com/goldbergonyi/nodebestpractices